the database. For example, you are guaranteed that `my_schema.some_field` won't result in a database query.

Change and Validate Data

Ecto changesets[8] cast and validate data from external sources into your database. Ecto provides a variety of default validation functions that let you check for the presence of data, ensure data consists of specific values, and more. Plus, you can always add application-specific custom validation.

We'll be diving deeper into each of these concepts throughout the rest of this chapter. Next, let's use Ecto to define our application's data structures!

Write an Ecto Schema

Databases revolve around data, so we need a way to define the types, tables, and relationships of our application's data layer. Ecto solves this with schemas. We use schemas to define our data structure, and then Ecto uses the schema definition to operate on our application's data.

We'll write a schema to hold SMS messages, and then we'll look at how Ecto fields and associations work. Let's jump in!

Phoenix Generators



Phoenix provides a generator that's used to define a schema file and migration file all at once. The reason why we don't use that in this book is simple: I don't like using most generators. I prefer the control of writing the code myself, and it gives me time to think about the schema I'm defining.

Many people like using generators because they allow you to quickly implement commonly repeated code templates. If you want to use the `mix phx.gen.schema` generator for your own projects, you can read its documentation.[9]

Define an SMS Message Schema

`Ecto.Schema` uses macros to create a domain-specific language (DSL) that we use to define our schema. This results in a data definition that's easy to read and write. Create a file at `lib/phone_app/conversations/schema/sms_message.ex` in the

8. https://hexdocs.pm/ecto/Ecto.Changeset.html
9. https://hexdocs.pm/phoenix/Mix.Tasks.Phx.Gen.Schema.html